

Next.js 7.2 — Authentication Architecture

Now we get into the **actual machinery**.

The key question is:

After I log in once, how does the server know who I am on my next request?

Because HTTP itself is basically:

```
Request → Response  
Request → Response  
Request → Response
```



Each request is independent.

So if you log in:

```
POST /login  
email = gopi@example.com  
password = ****
```



and then make another request:

GET /dashboard



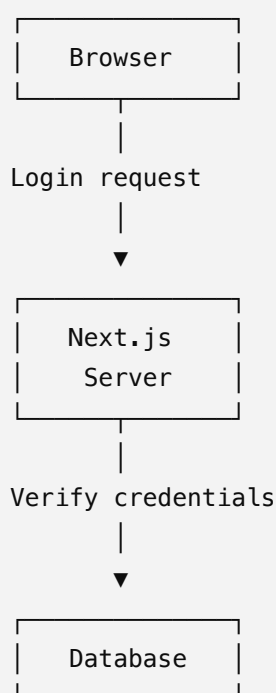
the second request doesn't automatically know that you're Gopi.

Something has to **carry your authenticated identity forward**.

That's where **sessions, cookies, and tokens** come in.

1. The big picture

A typical Next.js authentication architecture looks like this:



If credentials are valid:



Then later:



That's the core architecture.

2. Let's build the login flow step by step

Suppose we have:

```
/login
/dashboard
```

And Gopi enters:

```
Email: gopi@example.com
Password: myPassword
```

Step 1 — Browser sends login request

The browser sends something like:

```
POST /login
```

with:

```
{
  "email": "gopi@example.com",
  "password": "myPassword"
}
```

Important:

The browser does NOT decide whether the credentials are valid.

The server does.

3. Step 2 — Next.js server receives it

For example, we could use a Server Action:

```
"use server";

export async function login(formData: FormData) {
  const email = formData.get("email");
  const password = formData.get("password");

  // verify credentials...
}
```

Or a Route Handler:

```
app/api/login/route.ts
```



Both can participate in authentication.

4. Step 3 — Server checks the database

The server asks the database:

```
SELECT * FROM users
WHERE email = 'gopi@example.com';
```

Suppose the database returns:

```
id: 123
email: gopi@example.com
```



```
passwordHash: ...  
role: "user"
```

The server then verifies the password against the stored password hash.

If it matches:

```
Authentication successful 
```



Notice something important:

Passwords should not be stored as plain text.

You don't want:

```
password = "myPassword"
```



in your database.

Instead, you store a password hash and verify the supplied password against it.

We'll cover the security side later.

5. Step 4 — Server creates a session

Now the server needs to establish:

```
"This browser belongs to user 123."
```

There are different ways to do this.

The two big concepts we'll study are:

Option A — Server-side session

Database:

```
sessionId: abc123  
userId: 123  
expiresAt: ...
```



Browser gets:

```
sessionId = abc123
```



Option B — Token-based session

The server creates a token containing/representing information about the authenticated user.

For example, a JWT might contain claims such as:

```
{
  "sub": "123",
  "role": "user",
  "exp": 1790000000
}
```

We'll go deep into the differences in 7.3.

For now, remember:

A session/token gives the server a way to recognize the browser on future requests.

6. Step 5 — Server sends a cookie

This is where the browser gets something important.

The server can send:

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure
```

The browser stores that cookie.

Conceptually:

Browser



Cookies

sessionId = abc123

The user doesn't normally need to manually attach it to every request.

The browser handles that.

7. Step 6 — User requests /dashboard

Now Gopi navigates to:

/dashboard



The browser sends:

```
GET /dashboard
Cookie: sessionId=abc123
```

🎯 **This is the critical part.**

The browser is essentially saying:

"Here's my session identifier."

8. Step 7 — Server validates the session

Next.js can read the cookie.

Conceptually:

```
const cookieStore = await cookies();

const sessionId = cookieStore.get("sessionId");
```

Then the server can ask:

```
sessionId = abc123
  ↓
Find session
  ↓
userId = 123
  ↓
Find user
  ↓
Gopi
```



Now the server knows:

Authenticated user = Gopi



9. Step 8 — Authorization happens

Now suppose Gopi requests:

/admin



The server might determine:

User:

Gopi



Role:

user

Required:

admin

Therefore:

Authentication → 

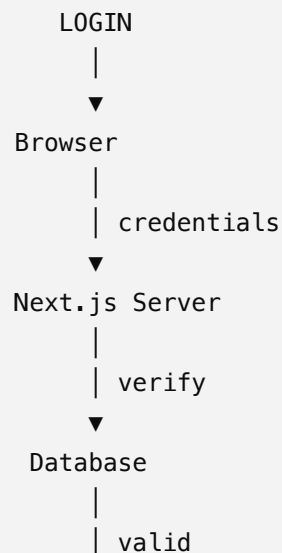
Authorization → 

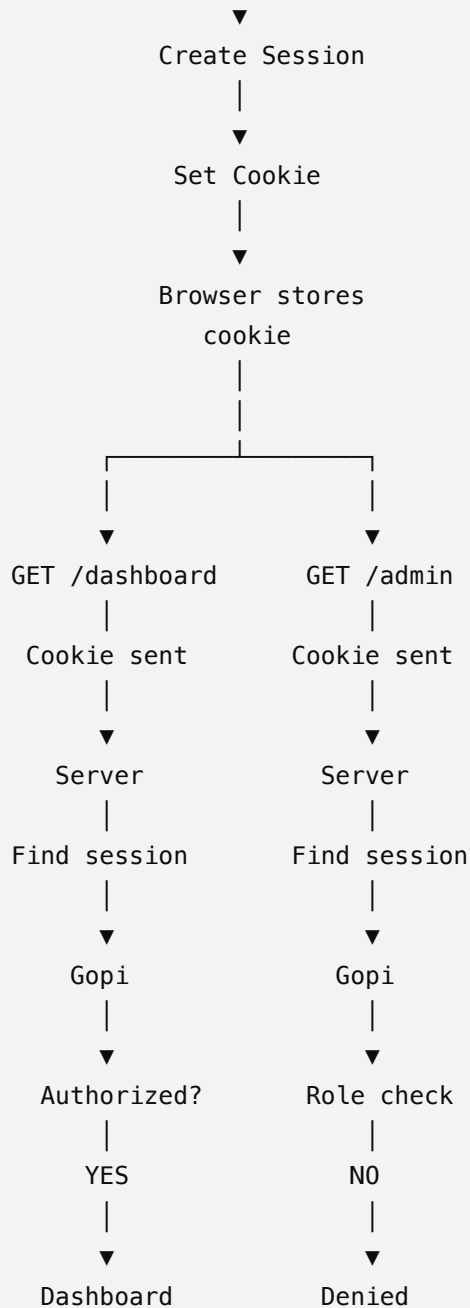


The server denies access.

10. Put the entire flow together

Here's the mental model I want you to remember:





🔥 That's authentication architecture in one picture.

11. Where does `proxy.ts` fit?

This is where our previous Next.js lesson comes back.

Suppose we have:

```

/dashboard
/settings
/admin
  
```

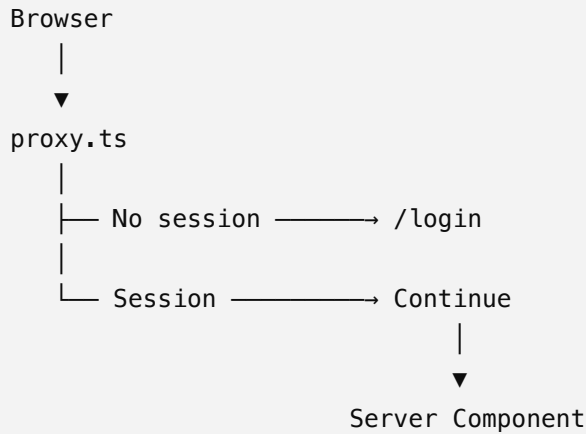


We don't necessarily want every request to reach the page before discovering:

"Oh, this person isn't logged in."

We can use `proxy.ts` for request-time checks/routing decisions.

Conceptually:



But here's an **important interview nuance**:

Proxy is useful for early request handling and redirects, but it should not be your only authorization/security boundary.

Your server-side data access and mutations should still verify authentication/authorization.

Why?

Because protecting a UI route doesn't automatically protect the underlying operation.

12. Example: Why server authorization still matters

Imagine:

`/admin/users`

Your proxy says:

```
if (!session) redirect("/login");
```

Great.

But now suppose there's an API/Server Action:

```
deleteUser(userId)
```



A malicious client might attempt to invoke the operation directly.

So the Server Action needs its own check:

```
"use server";

export async function deleteUser(userId: string) {
  const user = await getCurrentUser();

  if (!user) {
    throw new Error("Unauthorized");
  }

  if (user.role !== "admin") {
    throw new Error("Forbidden");
  }

  await deleteUserFromDatabase(userId);
}
```

This gives us:

```
proxy
  ↓
Early access control / redirect

Server Action
  ↓
Actual security enforcement

Database
  ↓
Data
```



13. One terminology detail you should know

You'll hear:

Authentication

```
AuthN
```



Authorization

AuthZ



So in interviews:

AuthN = identity

AuthZ = permissions

Easy mnemonic:

AUTHENTICATION

↓

N

Identity

AUTHORIZATION

↓

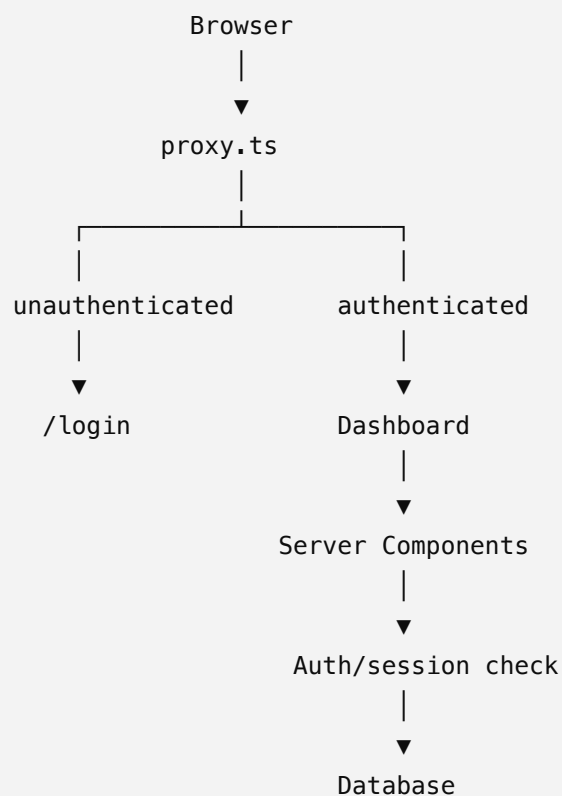
Z

Permissions

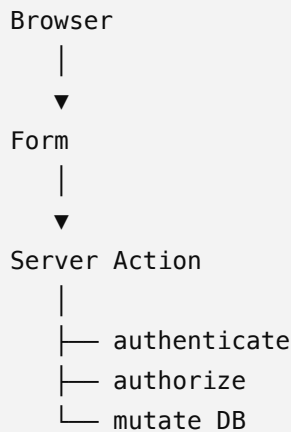


14. The architecture we'll eventually build

Our final application will look roughly like:



And for mutations:



That's the architecture you should have in your head.

7.2 Interview Question

"How does a web application remember that a user is logged in?"

A good answer:

After successful authentication, the server establishes a session, commonly represented by a session identifier or token. The browser stores the session information, typically in a secure cookie. On subsequent requests, the browser sends the cookie automatically, allowing the server to identify and validate the user's session and then perform authorization checks.

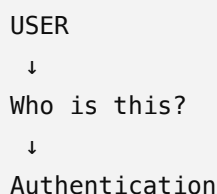
That's a much stronger answer than:

"We store the JWT in localStorage." 🤪

Because **where and how authentication state is persisted is an architectural/security decision**, not simply a storage choice.

One thing I want you to lock in before 7.3 🔒

Don't mix these three concepts:



SESSION

How do we remember them?



Session management

PERMISSION

What can they do?



Authorization

Next we'll tackle the part that causes a **LOT** of confusion:

7.3 — Cookies vs Sessions vs JWTs

We'll take one login request and trace **exactly what is stored in the browser, what is stored on the server/database, and what travels over the network** for each architecture.



okay then



Next.js 7.3 — Cookies vs Sessions vs JWTs

This is **one of those topics** where tutorials often blur three different things together. Let's separate them cleanly.

The first thing to understand:

Cookie, Session, and JWT are not three competing versions of the same thing.

They solve **different parts** of the authentication problem.

1. First: What is a Cookie?

A **cookie is storage + automatic transport** provided by the browser.

For example, the server can send:

```
</> http
```



```
Set-Cookie: sessionId=abc123
```

The browser stores it.

Later, when requesting your site:

```
</> http
```



```
GET /dashboard
```

```
Cookie: sessionId=abc123
```

The browser automatically sends the cookie.

So:

```
Cookie
```

```
↓
```

```
Browser mechanism
```

```
↓
```

```
Stores small pieces of data
```

```
↓
```

```
Automatically sends them with matching requests
```



Important

A cookie itself does **not** mean the user is authenticated.

You could have:

```
theme=dark
```



as a cookie.

That's obviously not authentication.

2. What is a Session?

A **session** represents the authenticated state of a user.

For example, your server/database could have:

```
Sessions table
```



```
|-----|
```

sessionId	userId	expiresAt
abc123	42	tomorrow

And the browser has:

Cookie:
sessionId=abc123



So when Gopi requests:

GET /dashboard



the browser sends:

Cookie: sessionId=abc123



Server:

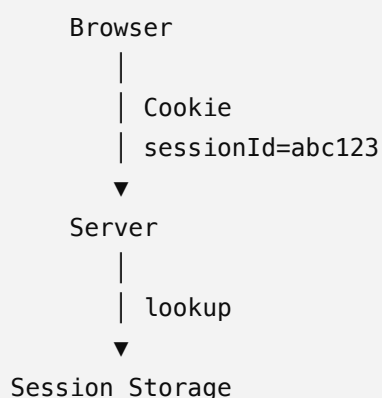
abc123
↓
Find session
↓
userId = 42
↓
Find user 42
↓
Gopi



That's a **server-side session architecture**.

3. So Cookie + Session often work together

This is the important relationship:



|
| userId=42
▼
User

So:

Cookie

Carries the identifier.

Session

Represents the authenticated state.

4. Now what is a JWT?

JWT = JSON Web Token.

Instead of giving the browser only:

abc123



the server creates a signed token containing claims.

Conceptually:

HEADER.PAYLOAD.SIGNATURE



The payload might contain:

```
</> JSON

{
  "sub": "42",
  "role": "user",
  "exp": 1790000000
}
```



The server signs it.

The browser then possesses that token.

5. Here's where people get confused 🤪

You can put a JWT **inside** a cookie.

For example:

Browser



Cookie

authToken = <JWT>

So:

Cookie ≠ JWT

A cookie is the **transport/storage mechanism**.

JWT is a **token format**.

You can have:

Cookie



└─ JWT

6. Three common architectures

Now let's compare them.

Architecture A — Server-side session

Browser:

Cookie:

sessionId=abc123



Server/database:

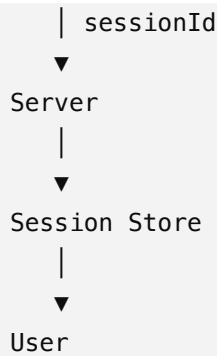
abc123 → userId 42



Flow:

Browser





The browser does **not** contain the complete session state.

It has an identifier.

7. Architecture B — JWT in a cookie

Browser:

Cookie:
authToken=<JWT>



JWT contains claims:

```
</> JSON
{
  "sub": "42",
  "role": "user",
  "exp": 1790000000
}
```



Server receives it:

```
JWT
↓
Verify signature
↓
Read claims
↓
userId = 42
```

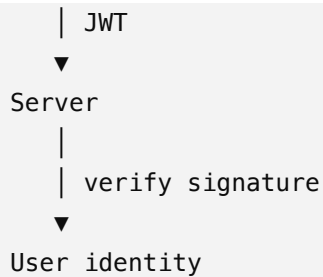


The server may not need to look up a session record for every request.

Conceptually:

Browser
|





8. Architecture C — JWT in localStorage

You'll also see:



```
Browser localStorage
  ↓
authToken=<JWT>
```

Then JavaScript reads it:



```
</> JavaScript

const token = localStorage.getItem("authToken");
```

and sends it manually:



```
</> http

Authorization: Bearer <JWT>
```

This is a different architecture.

And for browser-based authentication, storing long-lived auth tokens in `localStorage` has an important downside:

JavaScript running in the page can access localStorage.

So if an attacker manages to execute malicious JavaScript through an XSS vulnerability, that token can potentially be stolen.

That's one reason secure, `HttpOnly` cookies are commonly preferred for web sessions.

9. HttpOnly — very important

Suppose we have:

```
</> http
```

```
Set-Cookie: sessionId=abc123; HttpOnly
```



`HttpOnly` means browser JavaScript cannot read that cookie through:

```
</> JavaScript
```

```
document.cookie
```



But the browser can still send it with requests.

So:

JavaScript

|
|— document.cookie ❌
|

▼
Cookie inaccessible

Browser networking

▼
Cookie sent automatically ✅



This reduces the ability of an XSS payload to directly steal the cookie.

But: `HttpOnly` does not magically prevent XSS itself, and it doesn't make every request safe. Other protections, including appropriate CSRF defenses depending on the authentication setup, still matter.

10. Secure

Another cookie attribute:

```
</> http
```

```
Secure
```



means:

Send the cookie only over HTTPS connections.

So production authentication cookies commonly look conceptually like:

```
</> http

Set-Cookie:
sessionId=abc123;
HttpOnly;
Secure;
SameSite=Lax
```



We'll later break down every attribute.

11. Let's compare the architectures

	Server Session	JWT in Cookie	JWT in localStorage
Browser stores	Session ID	JWT	JWT
Server stores session state	Usually yes	Not necessarily	Not necessarily
Cookie involved	Usually	Often	✗
JS can directly read token	No with HttpOnly	No with HttpOnly	✓
Browser sends automatically	✓	✓	✗
Server lookup required	Usually	Often not for basic validation	Often not
Easy session revocation	✓	More complicated	More complicated

The important word is "usually".

Real systems can combine these approaches in different ways.

12. Here's a subtle but important point

People often say:

"JWT authentication is stateless."

That's **partly true, but context matters.**

A JWT can allow the server to validate identity without looking up a server-side session record on every request.

But your application may still have state elsewhere:

```
JWT
+
refresh token
+
database
+
revoked-token list
```



So don't blindly say:

"JWT means there is no server state."

Instead say:

A self-contained JWT can allow request authentication without requiring a server-side session lookup for every request, although the overall authentication system may still maintain server-side state.

That's a much more accurate interview answer.

13. Session vs JWT — the key conceptual difference

Imagine Gopi logs in.

Server session

Browser gets:

```
abc123
```



Server knows:

```
abc123 → Gopi
```



So:

```
Identifier → Server-side state
```



JWT

Browser gets:

```
JWT
```



JWT itself carries claims:

```
userId = 42  
role = user  
expiry = ...
```



and the server verifies its signature.

So:

```
Token → Claims + cryptographic verification
```



That's the core difference.

14. One thing that surprises people

"If JWT contains user information, can't the user change it?"

They can decode it.

JWT payloads are generally **encoded, not encrypted**.

For example:

```
</> JSON  
  
{  
  "userId": 42,  
  "role": "user"  
}
```



A user can inspect that.

They might try changing:

```
</> JSON  
  
{  
  "userId": 42,
```



```
"role": "admin"
}
```

But then the signature no longer matches.

Server:

```
Modified JWT
  ↓
Verify signature
  ↓
❌ Invalid
```

So the important security property is the **signature**, not secrecy of the payload.

If you need confidentiality, that's a separate concern from ordinary signed JWT integrity.

15. Now let's connect this to Next.js

Suppose you're inside a Server Component:

```
</> TypeScript

import { cookies } from "next/headers";

export default async function Dashboard() {
  const cookieStore = await cookies();

  const sessionId = cookieStore.get("sessionId");

  // validate session...

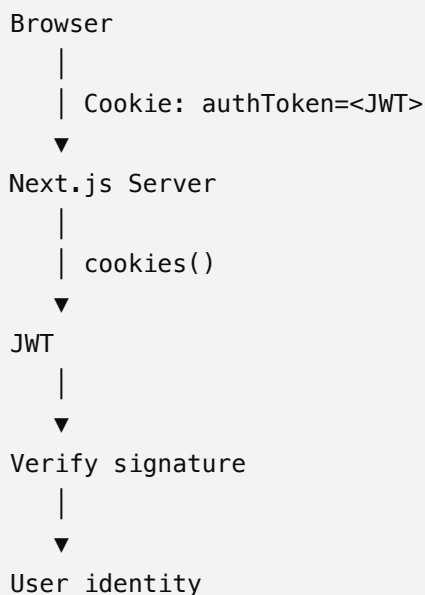
  return <h1>Dashboard</h1>;
}
```

The architecture could be:

```
Browser
  |
  | Cookie: sessionId=abc123
  ▼
Next.js Server
  |
  | cookies()
  ▼
sessionId
  |
```



Or if using a JWT cookie:



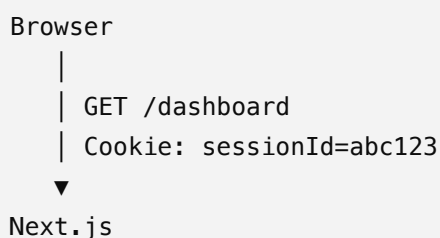
So now our earlier Next.js concepts start connecting.

16. What happens on every request?

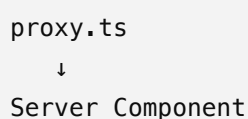
Let's take:

/dashboard

Request



Then potentially:



↓
Auth/session check
↓
Database
↓
Render

But remember:

Proxy is not the authentication system itself.

It can inspect the request and make routing decisions, while the actual server-side authorization checks should protect sensitive data and mutations.

17. One extremely important distinction for interviews

Cookie

A browser-managed mechanism for storing and sending small pieces of data with requests.

Session

Server-side or otherwise managed state representing an authenticated user's login state.

JWT

A signed token format containing claims that can be verified by the server.

Token

A general concept: some credential/value representing authorization or identity. A JWT is one type of token.

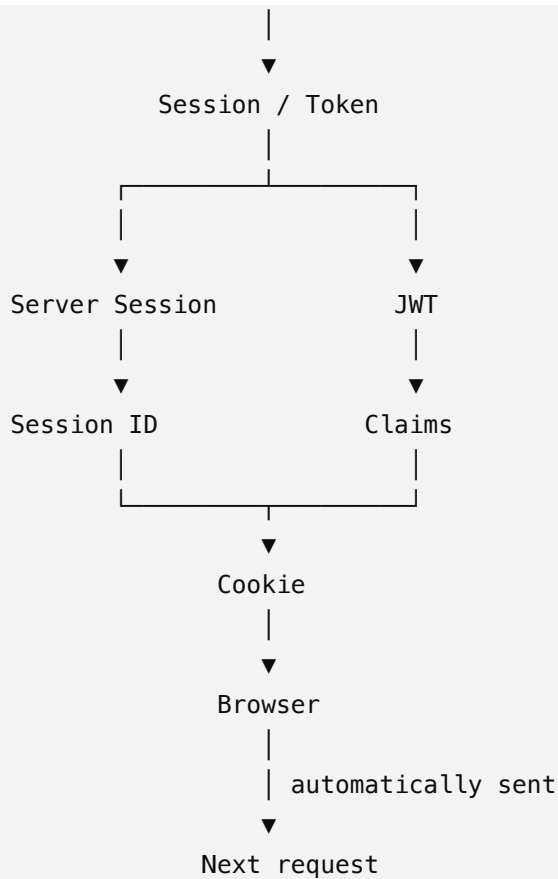


Your mental picture

Keep this picture in your head:

AUTHENTICATION
↓
User logs in





And **cookie** is not synonymous with session or JWT.

That's the big takeaway from 7.3. 🔥

🎯 Interview check

Before we move to 7.4 — Login Flow, answer these 5 questions yourself:

Q1.

Is a cookie the same thing as a session?

Q2.

Can a JWT be stored inside a cookie?

Q3.

Why is an `HttpOnly` cookie generally safer against token theft through XSS than putting the token in `localStorage`?

Q4.

In a server-side session architecture, what does the browser usually store?

- A. The entire user session
- B. A session identifier
- C. The user's password
- D. The entire database record

Q5.

If a JWT payload says:

```
</> JSON
{
  "role": "admin"
}
```

can the user simply edit it to `"role": "user"` or `"admin"` and have the server trust the modified value?

Answer these **without looking back** 😊 — then I'll correct them, and we'll move into the really useful part: **exactly what happens from clicking "Login" → setting the cookie → redirecting → loading a protected Server Component.**



=

Good catch 😊 . When I said "**another server endpoint**", I was using a broad term. In the Next.js context we're learning, you mainly need to distinguish **Route Handlers and Server Actions**.

What is a server endpoint?

An **endpoint** is a place on the server that can receive a request and do something.

For example:

```
POST /api/change-email
```



is an endpoint.

In Next.js, the most obvious example is a **Route Handler**:

```
app/  
└─ api/  
    └─ change-email/  
        └─ route.ts
```



</> TypeScript

```
export async function POST(request: Request) {  
  // server code  
}
```



So:

```
Browser  
  │  
  │ POST /api/change-email  
  ▼  
route.ts
```



That's a server endpoint.

What about Server Actions?

A Server Action is also server-side callable code:

```
</> TypeScript
"use server";

export async function changeEmail(formData: FormData) {
  // server code
}
```

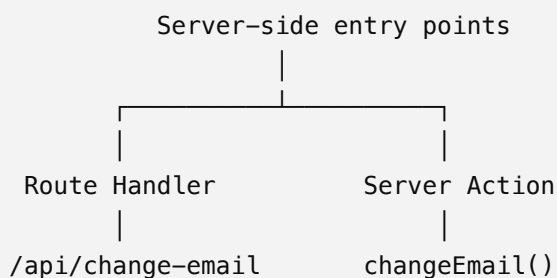
Called from:

```
</> TypeScript
<form action={changeEmail}>
```

Conceptually:

```
Browser
  |
  | invoke action
  ▼
Server Action
```

So for our purposes:



What else can exist?

Outside of Next.js-specific terminology, you can also have server endpoints implemented by:

Traditional backend

```
Express
Fastify
```

NestJS
Django
Spring Boot
Rails

For example:

POST /api/orders



handled by an Express server.

External services

Your Next.js application might call:

Stripe API
GitHub API
Google API



Those are also HTTP endpoints, but they're **external endpoints**, not endpoints belonging to your Next.js application.

And there's one more distinction

A **Server Component** is NOT an **endpoint** in the same sense.

For example:

```
</> TypeScript

export default async function Dashboard() {
  const user = await getCurrentUser();

  return <DashboardUI />;
}
```



It's server-side code, but it's a **component**, not a public API endpoint like:

POST /api/delete-user



Similarly, a utility function:

```
</> TypeScript

async function getCurrentUser() {
  ...
}
```

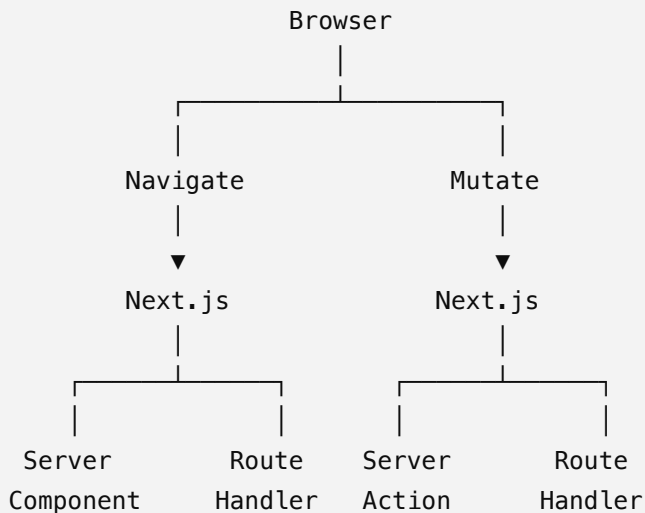


}

is server-side code but isn't necessarily an endpoint.

So for YOUR Next.js roadmap

You can simplify the whole thing to:



But the important security rule remains:

```

Route Handler
  ↓
Authenticate
  ↓
Authorize
  ↓
Do operation
  
```

and:

```

Server Action
  ↓
Authenticate
  ↓
Authorize
  ↓
Do operation
  
```

So when I said "another server endpoint," you can mentally replace that with:

"another server-side entry point, such as a Route Handler or Server Action."

That's the terminology we'll use going forward so I don't muddy the waters. 🍌



okay next

Perfect 😊 Let's continue.

🔒 Next.js 7.4 — Login Flow Step-by-Step

Now we'll connect everything from 7.1–7.3 into one real login flow.

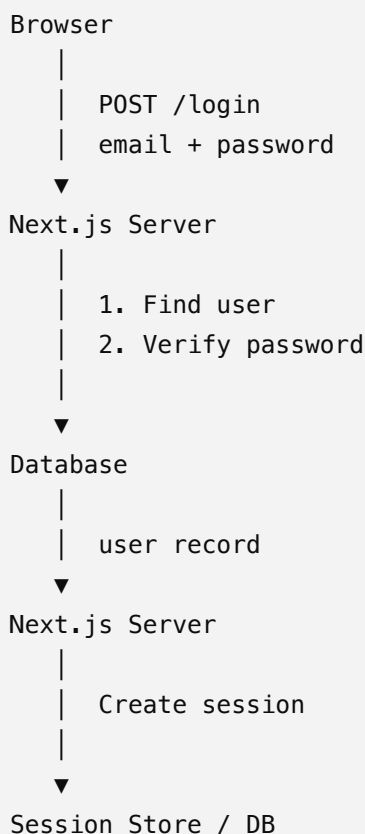
The goal is to understand exactly what happens when you enter:

```
email: gopi@example.com
password: *****
  ↓
Login
```



1. The big picture

A typical cookie-based login looks like:





Then every later request can look like:



That's the entire authentication mechanism at a high level.

2. What happens when the user submits Login?

Suppose we have:

```
</> TypeScript<br/><form action={login}><br/>  <input name="email" /><br/>  <input name="password" type="password" /><br/>  <button type="submit">Login</button><br/></form>
```

And:

```
</> TypeScript
```

```
"use server";

export async function login(formData: FormData) {
  const email = formData.get("email");
  const password = formData.get("password");

  // ...
}
```

When the user clicks **Login**:

```
Browser
  ↓
login Server Action
  ↓
Server
```



The important thing is:

The password verification happens on the server, not in the browser.

3. Server finds the user

The server might do:

```
</> TypeScript

const user = await db.user.findUnique({
  where: {
    email,
  },
});
```



Database:

```
users

id | email | passwordHash
---|-----|-----
42 | gopi@example.com | $2b$12$.....
```



Notice something important:

The database should NOT contain:

```
password = "mypassword123"
```



Instead it contains a **password hash**.

4. Server verifies the password

The server receives:

```
password = "mypassword123"
```



But the database has:

```
passwordHash = "$2b$12$...."
```



The server uses a password hashing library such as bcrypt/Argon2 to check:

</> TypeScript



```
const valid = await verifyPassword(  
  password,  
  user.passwordHash  
);
```

Conceptually:

Entered password



↓

hashing

↓

compare with stored hash

↓

match?

/

\

yes

no

If wrong:

</> TypeScript



```
return { error: "Invalid credentials" };
```

If correct:

Authentication successful



At this point we know:

This person successfully proved they know the password for this account.

That's authentication.

5. Now create a session

After successful authentication:

</> TypeScript

```
const sessionId = crypto.randomUUID();
```



Then the server stores something like:

sessions



sessionId	userId	expires
abc123	42	2026-09-24

So:

abc123 → user 42



This is **server-side session state**.

6. Server sends the session ID to the browser

The server sets:

</> TypeScript

```
cookieStore.set("sessionId", sessionId, {  
  httpOnly: true,  
  secure: true,  
  sameSite: "lax",  
  path: "/",  
});
```



The browser receives something conceptually like:

</> http



```
Set-Cookie: sessionId=abc123;  
           HttpOnly;  
           Secure;  
           SameSite=Lax
```

The browser stores it.

7. The browser does NOT need to manually attach it

This is a really important concept.

Suppose later the browser requests:

```
</> http  
  
GET /dashboard
```



The browser automatically sends:

```
</> http  
  
Cookie: sessionId=abc123
```



You don't normally write:

```
</> JavaScript  
  
fetch("/dashboard", {  
  headers: {  
    sessionId: "abc123"  
  }  
});
```



The browser's cookie mechanism handles it.

8. How does `/dashboard` know who Gopi is?

The dashboard Server Component can do something like:

```
</> TypeScript  
  
const cookieStore = await cookies();  
  
const sessionId =  
  cookieStore.get("sessionId")?.value;
```



Now:

```
sessionId = abc123
```



Server checks the session store:

```
abc123  
  ↓  
userId = 42
```



Then:

```
userId = 42  
  ↓  
users table  
  ↓  
Gopi
```



So the server can now say:

```
</> TypeScript  
  
const user = await getCurrentUser();
```



and get:

```
</> TypeScript  
  
{  
  id: 42,  
  email: "gopi@example.com",  
  role: "user"  
}
```



9. Then authorization happens

Suppose:

```
/dashboard
```



is available to all logged-in users.

Then:

</> TypeScript



```
if (!user) {  
  redirect("/login");  
}
```

Done.

But suppose:

/admin



requires an admin.

Then:

</> TypeScript



```
if (!user) {  
  redirect("/login");  
}  
  
if (user.role !== "admin") {  
  redirect("/forbidden");  
}
```

Notice the distinction:

Authentication

↓

Who are you?

↓

Gopi, userId=42

↓

Authorization

↓

What can Gopi access?

↓

admin? ❌



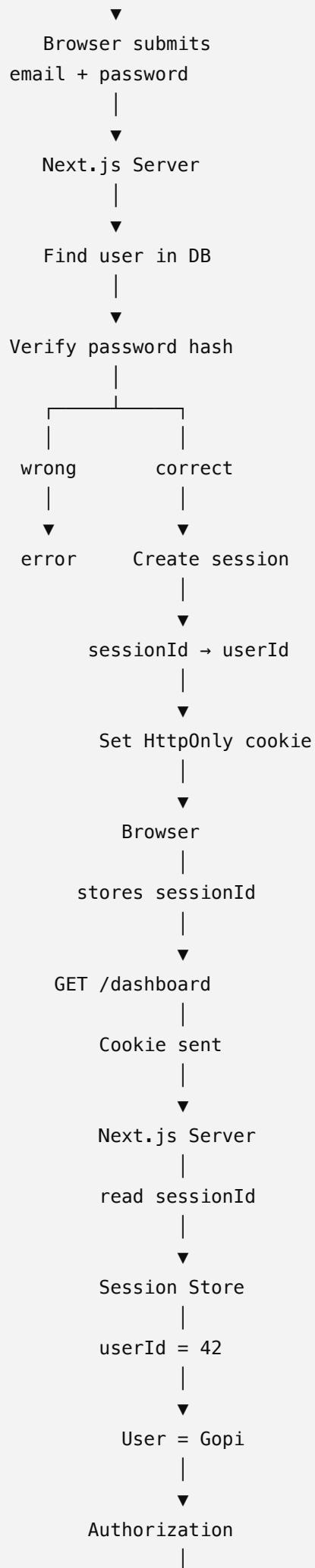
10. The complete flow

Put everything together:

LOGIN

|





Render page

Interview answer

If an interviewer asks:

"Explain the login flow in a Next.js application."

A strong answer would be:

"The client submits the credentials to the server, typically through a Server Action or Route Handler. The server looks up the user and verifies the submitted password against the stored password hash. If authentication succeeds, the server creates a session and sends a secure, HttpOnly cookie containing the session identifier to the browser. On subsequent requests, the browser automatically sends that cookie. The server uses the session identifier to retrieve the authenticated user and then performs authorization checks before allowing access to protected resources."

That's a **very solid mid-level answer**.

One subtle thing to remember

Don't think:

Cookie = user



Think:

Cookie
↓
session identifier
↓
session
↓
user



For a server-side session architecture.

And:

JWT cookie
↓
JWT



```
↓  
claims  
↓  
user identity
```

for a JWT-based architecture.

That distinction will become important when we get to **7.5 — Server Components + Auth**.



wait we will cont.this part tommorrow give me a prompt to add in new chat

Absolutely 😊 Tomorrow, start a new chat and paste this:

Next.js 7.4 continuation prompt



We are continuing my **Next.js learning roadmap**.

We are currently on **Lesson 7 — Authentication & Authorization**.

We completed:

- **7.1 Authentication vs Authorization**
- **7.2 Auth Architecture (Browser ↔ Server ↔ DB)**
- **7.3 Cookies, Sessions & JWTs**
- We also covered **XSS, CSRF, HttpOnly, Secure, SameSite, CSRF tokens, Route Handlers vs Server Actions**, and where authentication/authorization checks belong.

We just started **7.4 — Login Flow Step-by-Step**, but stopped midway.

Continue from **7.4**, not from the beginning.

The flow we were learning was:

Browser

- submit email/password
- Next.js Server (Server Action / Route Handler)
- find user in DB
- verify password hash

- create session
- store `sessionId` → `userId`
- set HttpOnly cookie
- browser automatically sends cookie on later requests
- server reads session
- identifies user
- performs authorization
- renders protected resource.

I want to learn this **from first principles**, with:

1. Real-world examples
2. Exact request/response flow
3. Practical Next.js code
4. Clear explanation of what happens in browser vs server vs DB
5. Common mistakes/confusions
6. Interview-ready answers
7. Comparisons where useful

Do NOT rush through the lesson. If something is commonly simplified in tutorials, explicitly tell me the simplified version vs the production-realistic version.

After finishing 7.4, continue with:

7.5 Server Components + Auth

then **7.6 Client Components + Auth**, etc., following the existing Lesson 7 roadmap.

That should drop us right back into the same spot tomorrow. 🙌🔥



ChatGPT can make mistakes. Check important info.